

Mind the Gap: An Empirical Study of Synchronization Gaps, Delays, and Missed Opportunities in Software Forks

JIAYING ZHU, Nanyang Technological University, Singapore

LYUYE ZHANG*, Nanyang Technological University, Singapore

JIAHUI WU, Nanyang Technological University, Singapore

CHENGYUE LIU, Nanyang Technological University, Singapore

YANG LIU, Nanyang Technological University, Singapore

Fork-based development enables parallel evolution of software, but unsynchronized contributions create persistent divergence: security patches, bug fixes, and quality improvements often fail to propagate across fork families, leaving downstream users exposed to known vulnerabilities or bugs and missing massive opportunities to improve the other repositories in the family. We present the first large-scale empirical study of fork synchronization, analyzing popular GitHub fork families with 3,820 actively maintained forks, and developed a monitoring platform to mine the valuable commits and promote their swift merging.

Our findings reveal a synchronization paradox: while 90% of submitted pull requests are merged, only 6.92% of fork commits ever appear in PRs, leaving massive fork development permanently unsynchronized across the families. Synchronization delay is pervasive and structurally uneven where fork propagation accounts for 72.9% of end-to-end commit lifecycle delay. Contrary to common assumptions, PR rejection is rarely caused by technical incorrectness; instead, 65% of rejections stem from superseded contributions, process violations, or maintainer policy decisions.

Based on these insights, we develop a three-stage syncability assessment pipeline that identifies fork-local commits that are both *sync-worthy* (broadly beneficial) and *sync-eligible* (technically and policy-compatibly portable). Applied to 0.5 million fork-local commits, our pipeline surfaces 12,284 sync-ready commit–repository pairs, demonstrating that our approach identifies practically valuable changes. To further validate the security impact, we manually reviewed 153 security-related commit–repository pairs and confirmed 83 as potential 1-day vulnerabilities, for which we produced 35 proof-of-concept of exploit demonstrations and filed issues to the affected repositories. Our monitoring platform enables continuous, near-real-time detection of synchronization opportunities across fork families, improving the sustainability of fork-based open-source ecosystems.

CCS Concepts: • **Software and its engineering** → **Software libraries and repositories; Software maintenance tools; Software configuration management and version control systems.**

Additional Key Words and Phrases: Software forks, fork synchronization, empirical study, pull requests, patch propagation, one-day vulnerabilities, mining software repositories

ACM Reference Format:

Jiaying Zhu, Lyuye Zhang, Jiahui Wu, Chengyue Liu, and Yang Liu. 2026. Mind the Gap: An Empirical Study of Synchronization Gaps, Delays, and Missed Opportunities in Software Forks. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA042 (October 2026), 24 pages. <https://doi.org/10.1145/3832133>

*Corresponding author.

Authors' Contact Information: [Jiaying Zhu, jiaying007@e.ntu.edu.sg](mailto:jiaying007@e.ntu.edu.sg), Nanyang Technological University, Singapore; Lyuye Zhang, zh0004ye@e.ntu.edu.sg, Nanyang Technological University, Singapore; Jiahui Wu, jiahui004@e.ntu.edu.sg, Nanyang Technological University, Singapore; Chengyue Liu, CHENGYUE001@e.ntu.edu.sg, Nanyang Technological University, Singapore; Yang Liu, yangliu@ntu.edu.sg, Nanyang Technological University, Singapore.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/10-ARTISSTA042

<https://doi.org/10.1145/3832133>

1 Introduction

Fork-based development enables parallel evolution [12]: forks can rapidly prototype features, adapt to local constraints, and explore alternative designs while still benefiting from a shared upstream codebase. Multiple forks along with the original repository can form a fork family that retains substantial code overlap by sharing a common code base. However, parallelism also introduces a fundamental drawback, namely, unsynchronized contributions. When useful changes fail to propagate within a fork family, projects diverge in behavior and quality and may retain known defects or vulnerabilities long after they are addressed upstream. For example, Vim [56] and Neovim [42] (a hard fork [47] of Vim) have repeatedly shared code paths that led to command execution vulnerabilities: CVE-2019-12735 [40] affected both Vim [57] and Neovim [43], requiring coordinated fixes across the fork family, illustrating how downstream forks can remain exposed until patches are ported and released. Beyond security patches, failure to synchronize bug fixes and performance improvements can similarly compromise the quality and reliability of other repositories within the fork family.

This parallel-development setting is prone to inefficiencies [66], including lost or duplicate contributions and fragmented communities, making cross-repository synchronization a recurring challenge rather than an exceptional event [65]. The stakes are especially high for security: *one-day vulnerabilities* can persist in forks after the origin is patched because fixes do not reliably propagate at the commit level [31, 47]. Accordingly, existing work has largely studied synchronization through a security-centric lens (e.g., accelerating security fix propagation [35] and measuring patch diffusion delays in downstream ecosystems [1, 60, 64]). However, it remains unclear how broadly useful general changes (bug fixes, quality improvements, tests/docs) synchronize at scale and how synchronization lifecycle and lag shape what ultimately propagates, let alone how to automatically facilitate the synchronization among fork families regarding all useful changes.

To bridge this gap, we conduct the first large-scale empirical study to characterize synchronization in popular GitHub repositories by explicitly identifying fork families and quantifying their unsynchronized contributions. A central motivation is that synchronization gaps are typically invisible to downstream users: a fork may have been actively maintained at adoption time and only later stopped tracking upstream fixes, leaving users exposed without any warning signal. Our goal is therefore visibility into these latent gaps, not an assumption that all forks remain maintained indefinitely. Building on this evidence, we move beyond problem identification to ask how synchronization can be facilitated in a manner that is accurate, automated, and timely. Our final deliverable is a continuous, near-real-time platform that detects syncable commits and operationalizes their propagation (e.g., via actionable recommendations or PR creation), thereby improving the sustainability of fork-based open-source communities.

Achieving this goal requires capabilities that prior work has not yet delivered. First, existing studies often focus on domain-specific ecosystems (e.g., Linux vendors or blockchain forks), whereas we target general-purpose GitHub repositories at scale. Second, a fine-grained understanding of already synchronized contributions at the commit level—particularly their intent and timeliness—is limited, leaving the lifecycle of synchronization insufficiently explained. Third, the reasons why synchronization attempts fail have not been systematically studied: rejected fork→origin PRs and their rejection rationales remain under-explored. Finally, no prior approach provides a real-time detector for syncable commits using fine-grained criteria that jointly consider technical feasibility, repository policies, and practical usefulness, which are all necessary to avoid noisy or low-value synchronization attempts.

We addressed these gaps via four research questions and built an end-to-end monitoring platform based on the findings. RQ1 characterizes synchronization pathways and reveals the synchronization gaps among the repositories in the fork family. RQ2 quantifies synchronization lag in both fork→origin and origin→fork directions, clarifying where delay accumulates and measures the propagation life cycle of commits from creation to family-level synchronization; RQ3 analyzes what kinds of contributions synchronize, and why some are rejected, empirically yielding generally beneficial types; and RQ4 shifts to fork-local native commits, evaluating which changes are both *sync-worthy* and *sync-eligible* yet remain unsynchronized to promote effective synchronization. Building on these results, we develop a monitoring platform that tracks syncable commits across fork families, assesses their synchronizability conservatively, and can automatically create PRs to facilitate safe and low-friction synchronization. To validate the security impact, we manually reviewed 153 security-related commit–repository pairs and confirmed 83 as potential 1-day vulnerabilities where the target repository had not applied the fix, producing 35 proof-of-concept demonstrations and filing issues to draw maintainer attention. Our contributions are threefold:

- We are the first to quantify fork-family synchronization as a staged process that jointly models exposure, rejection rationales, and bidirectional lag, and then operationalize commit-level detection with policy/usefulness screens.
- We reveal what kinds of changes are successfully synchronized and why synchronization attempts fail, thereby providing actionable insights on how synchronization can be facilitated.
- We develop a monitoring platform that continuously tracks sync-ready commits in near real time and evaluates their synchronization potential using fine-grained criteria that jointly consider technical feasibility, practical usefulness, and policy compatibility with target repositories. We further validate the security impact by manually confirming 83 potential 1-day vulnerabilities out of 153 security-related pairs, producing 35 proof-of-concept demonstrations.

2 Background and Motivating Example

2.1 Background

Modern open-source development frequently involves parallel development across an origin repository and a large number of its forks. In the GitHub ecosystem, a fork is initially created as an exact copy of its origin repository, but it may evolve in one of two very different directions [65]. Some forks are short-lived and exist only to stage a pull request (PR) [20]. Others continue to develop independently, introducing new features, refactoring, or patches that never return to the origin. These “hard forks” often emerge when contributors face obstacles such as unresponsive maintainers or rejected pull requests [66]. Conversely, origin maintainers also introduce updates, such as bug fixes or security patches, that may never reach the forks. This bidirectional flow of commits creates a poorly understood ecosystem where important changes may fail to propagate.

In this ecosystem, we consider three primary entities:

- **Origin repository:** the canonical upstream project from which the forks are created.
- **Forks:** developer- or organization-owned copies. A fork may be a short-lived PR-staging copy, a passive mirror, or an independently evolving code line. Our study focuses on independently evolving forks with native post-fork commits, since these forks can diverge from the origin and create synchronization gaps.
- **Commit flows:** bidirectional synchronizations between origin and forks, either through PRs, cherry-picks [9], or manual porting.

An ideal ecosystem maintains a healthy synchronization: forks contribute improvements upstream, and upstream fixes propagate downstream. However, in practice, many commits, especially

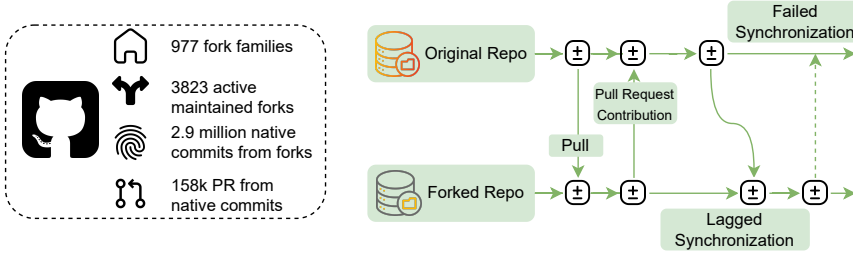


Fig. 1. Scenario Illustration of Commits Synchronization Between the Origin and Fork

critical ones—remain unsynchronized. As forks and origins continue diverging, incompatibilities increase, and important updates may be silently missed.

When commits fail to propagate, projects accumulate the gaps of changes: the set of commits that should have been adopted by another repository but were not. The gap is particularly harmful when the missing commits include security patches, correctness fixes, or compatibility adjustments. Downstream users of stale forks may unknowingly rely on vulnerable or incorrect code, while upstream maintainers may miss improvements already implemented in the forks.

2.2 Motivating Example

TARmageddon (CVE-2025-62518, CVSS 8.1) [41] is a boundary-parsing flaw in the async Rust tar ecosystem that enables archive-entry smuggling and can lead to remote code execution in downstream applications that unpack untrusted tar files [30]. Crucially, the vulnerability spans a fork family rooted at `async-tar` [3] (a Rust library for asynchronous tar archive I/O) and includes its derivatives. The most widely depended-on fork, `tokio-tar` [55], was effectively abandoned and would not receive a patch, forcing downstream users to migrate to a maintained fork, `astral-tokio-tar` [2] (a recent fork maintained by the Astral team, fixed in v0.5.6), rather than simply “pulling” upstream fixes [15, 45]. This incident illustrates the core risk we study: even when a fix exists within the fork family, unsynchronized or unmaintained forks can strand large downstream populations on vulnerable code. The fundamental difficulty is visibility: `tokio-tar`’s drift from upstream was not announced or signposted to its downstream users; many continued depending on it after maintenance had silently lapsed. Our goal is therefore to surface such latent synchronization gaps as early as possible so that downstream users and maintainers can make informed decisions, rather than to assume that all forks remain maintained indefinitely.

These observations motivate a systematic investigation into how often forks evolve independently, how contributions flow between origins and forks, what types of improvements arise in this parallel development, and how much technical lag accumulates across the ecosystem.

3 Data Collection

To construct a representative dataset for analyzing the technical lag phenomenon in fork ecosystems, we first collected the top open-source repositories from GitHub ranked by the number of stars. To ensure the dataset focuses on software projects, we filtered out non-code repositories using keyword-based heuristics (e.g., excluding dataset mirrors, tutorials, and pure documentation projects). After filtering, we retained 2,000 origin repositories. These popular projects typically exhibit active maintenance and extensive forking activity, making them suitable for studying contribution flows between origins and forks.

Sample-selection rationale. We select popular origins, ranked by stars, for two reasons. First, this choice follows prior large-scale GitHub fork studies [65, 66], which also focus on repositories with substantial public activity. Second, popular repositories are more likely to have active fork ecosystems and established contribution workflows, making cross-repository synchronization both observable and practically relevant. At the same time, star-based selection can introduce popularity bias, as noted in OSS sampling and ranking studies [6, 7, 36, 39]. We therefore make this criterion explicit and account for its implications in the threats-to-validity discussion. The resulting scale of 2,000 origins is comparable to or larger than prior large-scale fork studies [65, 66], which typically analyze hundreds to low thousands of repositories, while remaining feasible for exhaustive downstream fork collection and LLM-assisted labeling.

This sample yields 3,820 actively maintained forks and 2.9 million post-fork commits, which is sufficient to expose long-tailed ecosystem behavior without sub-sampling. We also considered initial selection by fork-graph complexity, such as fork count or fork-network depth, but did not use it as the sampling criterion. Fork-graph complexity is a downstream property of an origin: it can only be measured after retrieving and traversing the full fork network for each candidate, so it cannot guide the initial selection without first running the same collection pipeline. In our final sample, the 2,000 star-ranked origins yield 5.8 million forks (mean 2,901 forks per origin) and an average of 3.9 actively maintained forks per origin, confirming that this end of the distribution is well represented.

We first examined the prevalence of parallel development among all forks identified in our dataset. Across the 2,000 origin repositories, we collected 5,801,648 forks in total. From each selected repository, we retrieved all publicly available forks via the GitHub REST and GraphQL APIs [21, 28]. To ensure the analysis focused on software projects rather than documentation or configuration repositories, we heuristically filtered out non-code repositories by keywords such as dataset mirrors, tutorials, and pure documentation projects. After this filtering step, we obtained a total of **2,900,824** forks. We focus on direct forks of each origin rather than nested forks (forks of forks). This choice is well-supported: GitHub’s fork network architecture links all forks to a shared root repository, and the standard pull request workflow directs contributions to the upstream origin rather than intermediate forks [26]. Prior empirical studies of fork ecosystems similarly analyze direct origin–fork relationships, as nested forking is rare in practice and typically reflects transient experimentation rather than sustained parallel development [65].

As we aim to study the synchronization among origin repository and active forks, it is necessary to avoid analyzing the temporary forks that are only used to create PRs to the origin repository. To achieve this, we applied filters to only count those still under active maintenance. We applied three filtering criteria to identify actively and independently developed forks. ① The repository must have been updated with new commits within the 12 months preceding our data-collection snapshot in 2025, ensuring that it is currently maintained. ② The repository must have at least ten stars; prior work has shown that temporary or PR-only forks typically attract very few stars, whereas star count serves as a reasonable proxy for project popularity and sustained usage [36]. ③ The repository must contain at least 100 commits introduced after forking that do not originate from the upstream project, following the criteria from [65]. These commits are named as *native commits*. This criterion ensures that the fork exhibits independent parallel development, as a single commit may correspond to an isolated contribution, while multiple new commits indicate sustained development beyond the original codebase.

After applying the criteria, we retained **3,820** active forks, representing only **0.08%** of all forks. Due to the removal of forks, the final number of origin-fork clusters dropped to 977. On average, each origin repository had **3.9** actively maintained forks. These results demonstrate that while the vast majority of forks on GitHub are temporary or inactive, a non-negligible subset exhibits

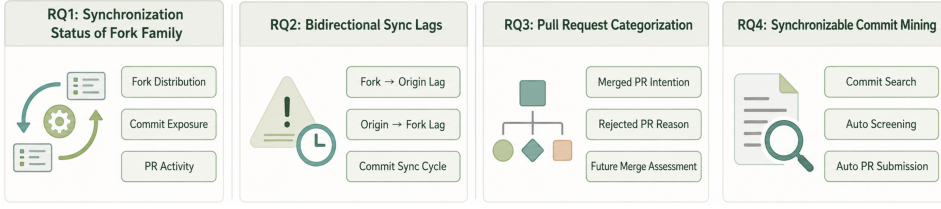


Fig. 2. Overview of Research Questions

sustained, independent development and thus has significant potential to diverge from their origins. The forks removed by these criteria are predominantly PR-only social forks, mirrors, and lightly modified tracking copies whose history is dominated by upstream pulls rather than native commits. They are out of scope because our study targets synchronization gaps arising from parallel evolution rather than passive upstream tracking, and they are therefore excluded from the statistics reported throughout the paper.

For each remaining origin–fork cluster, we then collected all commits and PRs. Specifically, we extracted (1) all merged PRs submitted to the 977 origin repositories (3.0 million in total) and (2) all commits from both the origin and its active forks. In total, this process yielded 25 million commits in the origin repositories and 2.9 million unique commits across the corresponding forks. There were 158,482 PRs created from the forks to the origin repositories that were derived from unique commits. These data form the empirical foundation for our subsequent analysis of contribution flows and technical lag within the fork ecosystem.

4 Empirical Study

4.1 Research Questions

This section organizes our empirical analysis around the paper’s overarching objective: facilitating the synchronization of useful changes within fork families so that parallel development remains sustainable rather than fragmenting into long-term divergence. First, we establish the current synchronization status of fork families and quantify the gap between fork-local development and cross-repository integration (**RQ1**). Second, recognizing that synchronization is only beneficial if it is timely, we measure how delay accumulates along the synchronization lifecycle in both fork→origin and origin→fork directions (**RQ2**). Third, to understand how synchronization can be improved, we analyze what kinds of changes successfully synchronize and why attempts fail, by characterizing the intentions of merged contributions and the rejection rationales of failed ones (**RQ3**). Finally, building on these insights, we operationalize a monitoring platform that continuously surfaces sync-ready fork-local commits and evaluates their synchronizability under conservative criteria (**RQ4**).

- **RQ1: Synchronization status.** How much development in fork families is synchronized?
- **RQ2: Synchronization timeliness.** When synchronization occurs, how long does it take, and where does the delay accumulate over the lifecycle?
- **RQ3: What syncs and what fails.** What kinds of changes are merged, and why do synchronization attempts get rejected?
- **RQ4: Missed synchronization opportunities.** Which fork-local unique commits are both sync-worthy and sync-eligible yet remain unsynchronized, and what opportunities do they represent for improved collaboration?

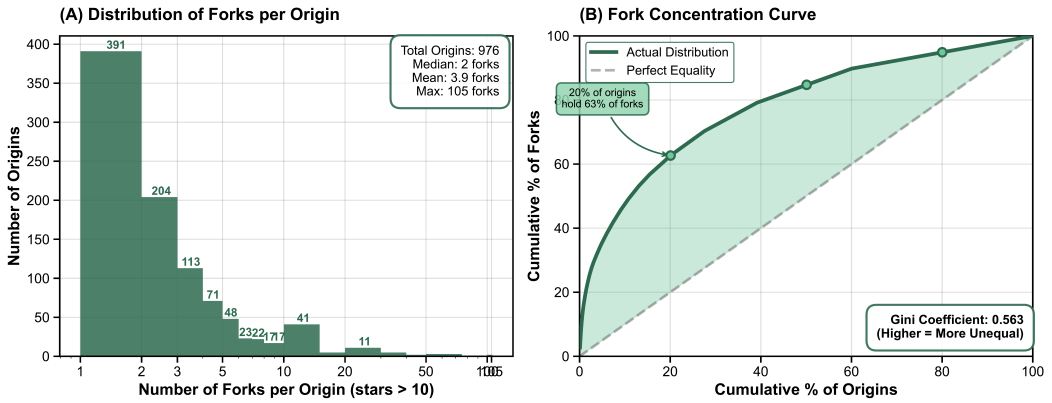


Fig. 3. Fork distribution and concentration across origin repositories. (A) Distribution of the number of active forks per origin (log-scale on the x-axis). (B) Fork concentration curve (Lorenz-style), showing strong inequality in fork activity.

4.2 RQ1: Prevalence of Parallel Development

4.2.1 RQ1.1: Fork Distribution and Concentration. **Fork activity is concentrated in a small subset of origins.** Across 977 origin repositories, we identified 3,820 active forks, with 1–207 forks per origin. Figure 3B quantifies the imbalance using a Lorenz-style concentration curve: the top 20% of origins account for approximately 63% of all active forks, yielding a Gini coefficient of 0.563. Figure 3A shows the corresponding long-tailed distribution from the per-origin angle: most origins host one or two active forks, while a small subset hosts dozens. Intensive fork-based development is therefore a property of a small fork-dense subset of origins rather than of the ecosystem at large.

This concentration has direct consequences for synchronization. The coordination workload within a fork family scales with its size: more parallel development branches mean more redundant fixes and more opportunities for commit staleness. In the fork-dense subset (e.g., Linux kernel variants and OpenSSL distributions), one well-propagated fix can reach hundreds of downstream repositories, whereas the origin with one or two forks faces a much lighter burden. Ecosystem-wide averages therefore understate the workload. We accordingly conduct our subsequent analyses at the per-family level and direct monitoring and tooling effort to the fork-dense families.

Finding 1: Fork activity is concentrated in a small fork-dense subset of origins: the top 20% of origins account for over 60% of all active forks (Gini = 0.56), with a median of 8 active forks per origin in the top quintile (maximum = 105). Synchronization pressure scales with family size and is amplified in this dense subset.

4.2.2 RQ1.2: The Synchronization Paradox—High PR Activity, Low Commit Exposure. RQ1.2 is scoped to fork→origin exposure because PR submission is the canonical observable pathway for forks to expose changes upstream; origin→fork synchronization proceeds through pulling, merging, or manual porting rather than PRs, so a symmetric PR-exposure metric is not meaningful in that direction. We therefore measure exposure at one direction here in RQ1 and revisit both directions at the lag level in RQ2, where shared-commit history signals allow direct comparison.

At first glance, PR-based synchronization appears healthy: across our dataset, forks submit pull requests at scale, with the 2,000 origin repositories receiving over 3 million PRs in aggregate. These submitted PRs enjoy a high merge success rate of 90.11%, suggesting that maintainers are receptive

to fork contributions. However, this apparent success masks a deeper problem: PR-level metrics capture the success of synchronization attempts once they are made, but they do not reflect how many native commits are covered in the PR events.

Measuring commit exposure. To quantify actual synchronization coverage, we define the **Commit Exposure Ratio (CER)** as the fraction of post-fork commits that appear in at least one pull request submitted to the origin:

$$\text{CER} = \frac{\text{\#post-fork commits appearing in PRs}}{\text{\#post-fork commits}}.$$

Counting commits toward CER: original commits and history-rewriting-aware commits.

The CER counts a post-fork commit as exposed only when its exact SHA appears in the commit metadata of at least one fork→origin PR. However, this measure may miss the SHA-changed commit when a contributor rebases or amends the commit before submission, since rebase and amend produce a different SHA for content-equivalent code.

To capture such re-submissions we extend the CER with a three-step classification pipeline. For each commit in a fork→origin PR we (1) check whether its SHA appears in the fork's commit set and label it *original* if so; (2) otherwise compare its diff against every fork unique commit (a commit in the fork but not in the origin) and label it *derivative* if the diff similarity is at least 85% or one diff's line set is a subset of the other's; (3) otherwise check whether its SHA appears in the origin and label it *from-origin*. Diff similarity is the Jaccard similarity of the added + removed line sets of the two unified diffs, after normalizing whitespace and stripping diff metadata (file headers, hunk markers, and line numbers); the 85% threshold and the subset rule are evaluated on the same line-set representation.

The history-rewriting-aware CER counts both *original* and *derivative* commits as exposed, while the original CER counts only *original*. An exact-SHA-only baseline yields a more conservative median CER of 6.92% (mean 18.38%); the rewriting-aware variant we adopt as the canonical CER for the rest of this paper raises these to 7.19% and 22.78% respectively.

Despite the high PR merge rate, commit exposure is therefore strikingly low: the median fork exposes only **7.19%** of its post-fork commits through PRs (mean: 22.78%). In other words, for a typical fork, more than 90% of commits never enter the upstream review process. This low exposure is pervasive: **66.17%** of all fork commits never appear in any PR, and in **86.2%** of fork–origin pairs, more than half of fork commits remain fork-local.

Figure 4 visualizes this disconnect: although many forks submit hundreds of PRs, the majority lie well below 50% commit coverage (75% of pairs have CER below 27.59%). This paradox persists regardless of project popularity—while popular origins attract more PR activity ($\rho = 0.29$), popularity shows almost no correlation with CER ($\rho = 0.07$). Among origins with over 10K stars, 51% have CER below 10%, confirming that ecosystem prominence drives contribution attempts but not comprehensive synchronization.

In absolute terms, nearly **3 million commits** across the dataset remain fork-local, never appearing in any PR to the origin. We admit that many such commits are intentionally fork-specific, reflecting customization, experimentation, or divergent project goals. Nevertheless, given the sheer volume of post-fork development, even a small fraction of sync-worthy changes within this large fork-local pool would represent substantial missed synchronization opportunities.

Finding 2: While submitted PRs are merged at a high rate (90%), most fork development never reaches the PR stage at all, the median fork exposes only 7.19% of its commits. This selective exposure raises critical concerns: unsynchronized commits may include security patches, bug

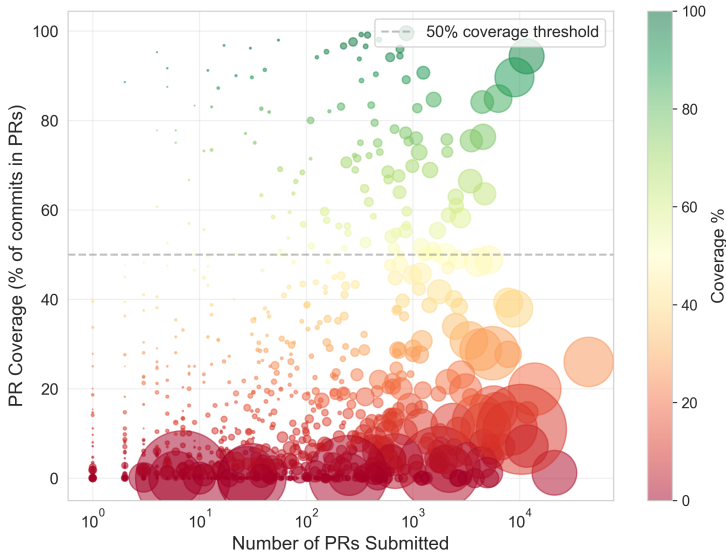


Fig. 4. The synchronization paradox: PR activity versus commit-level coverage. Each point represents a fork–origin pair. Bubble size indicates the total number of fork commits.

fixes, or valuable features that remain siloed in individual forks, potentially leaving downstream users exposed to known vulnerabilities.

4.3 RQ2: Synchronization Lag and Temporal Fragmentation

RQ1 reported a high merge rate (90%) for submitted PRs, but acceptance does not imply timeliness. Substantial delays may arise even for PRs that are ultimately merged, and PR review covers only one of three temporal stages in the end-to-end synchronization lifecycle of a commit. RQ2 therefore shifts the focus from whether fork-local commits reach the origin to when they do, in which stage of the lifecycle delay accumulates, and how long an integrated change takes to propagate back across the fork family.

We model fork-family synchronization as a three-phase pipeline spanning two units of observation as in Figure 5. Two of the phases characterize the fork→origin pathway and are defined entirely with respect to a single PR object: all timestamps in these definitions originate from the same PR, and the two phases sum to the per-PR fork→origin lag. The third phase characterizes another direction, the origin→fork pathway, in which synchronization occurs through fork-side pulling rather than through PR submission; its unit of observation is therefore a commit-fork pair.

- **Fork→Origin synchronization** is observable at the PR level, since the canonical contribution channel from a fork to its origin is the submission of a PR for review and merging by the origin’s maintainers. The lag of an individual PR decomposes into two sequential phases, each defined with respect to events (e.g., creation, merging) of the same PR (see Fig. 5):
 - **Developer Batching Phase.** For a given PR, the elapsed time between the authoring timestamp of the last fork commit and the creation timestamp of that PR. This phase quantifies developer-side batching and local iteration before upstream submission; batching lag is determined per the criterion under *Lag Determination* below.

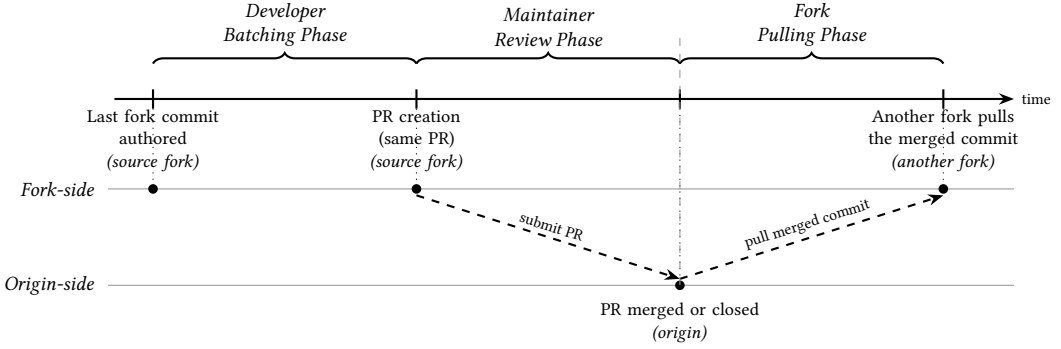


Fig. 5. Synchronization timeline across the three lag phases. The dashed vertical separator at PR merged or closed marks the directional boundary between Origin-Fork and Fork-Origin flows.

- **Maintainer Review Phase.** For the same PR, the elapsed time between the PR’s creation timestamp and its closure timestamp (whether merged or closed without merging). This phase quantifies maintainer-side review and decision latency; review lag is determined by the same criterion. Both phases are defined over the same PR, so their durations sum to the per-PR fork→origin lag.
- **Origin→Fork synchronization,** by contrast, is realized when a fork pulls commits from the origin rather than through PR submission. Here “pulling” refers to the fork-sync operations documented by GitHub [19] (“Sync fork” in the web UI and the equivalent `git fetch/merge/rebase` from upstream). Other forms of propagation, such as manual porting, reimplementation, or propagation through an intermediate repository, cannot be identified reliably from public Git/GitHub metadata and are outside this measurement. The corresponding lag is captured as the **Fork Pulling Phase**, defined as the elapsed time between the authoring timestamp of an upstream commit in the origin and the timestamp at which a given fork incorporates that commit into its own history. Since pulling lacks an associated PR object, the unit of observation for this phase is a commit-fork pair; pulling lag is determined by the same criterion.

Lag Determination. Rather than treating raw durations as “delay” in absolute terms, we define *lag* relative to the empirical baseline of each stage: for every time interval category (e.g., Developer Batching), we compute the average duration across all observations in that interval for the corresponding stage (a single PR for the two per-PR phases, a single commit-fork pair for Fork Pulling), and we label an observation as exhibiting lag only when its duration exceeds this interval-specific average. This normalization controls for temporal shifts in project activity and review workload, and ensures that “lag” reflects unusually slow progression compared to contemporaneous practice.

4.3.1 RQ2.1: Synchronization Lag in Fork↔Origin Flows. Table 1 summarizes the prevalence and severity of each lag component, while Figure 6 contrasts the *total* synchronization delay.

Table 1 indicates that lag is not rare and is unevenly distributed across stages. For fork→origin PRs in our dataset, **Review Latency Lag** (13.7%) and **Submission Batching Lag** (9.1%) occur at broadly comparable rates; we report these descriptively and do not claim one is a more prevalent bottleneck than the other. Moreover, **Fork Pulling Lag** is the most prevalent signal (59.3%), reflecting both the much higher volume of downstream pulling activities compared to selective upstream PR submissions, and the fact that a large fraction of forks operate under non-trivial origin-fork desynchronization. In terms of severity (among lagging cases), batching and review

Table 1. Overview of synchronization lag characteristics. Lag is reported only when the corresponding duration exceeds the empirical average for that stage.

Lag Type	#PRs/Forks	Prevalence	Median	Mean	P75	P90	Max
Delay (days)							
Fork→Origin (PR-level, $n = 158,485$)							
Submission Batching	14,428	9.1%	32.0	120.6	107.3	311.6	17,761
Review Latency	21,756	13.7%	46.4	137.1	131.9	368.3	4,228
Origin→Fork (fork-level, $n = 3,820$)							
Fork Pulling	93,972	59.3%	6.8	11.5	15.6	28.9	714

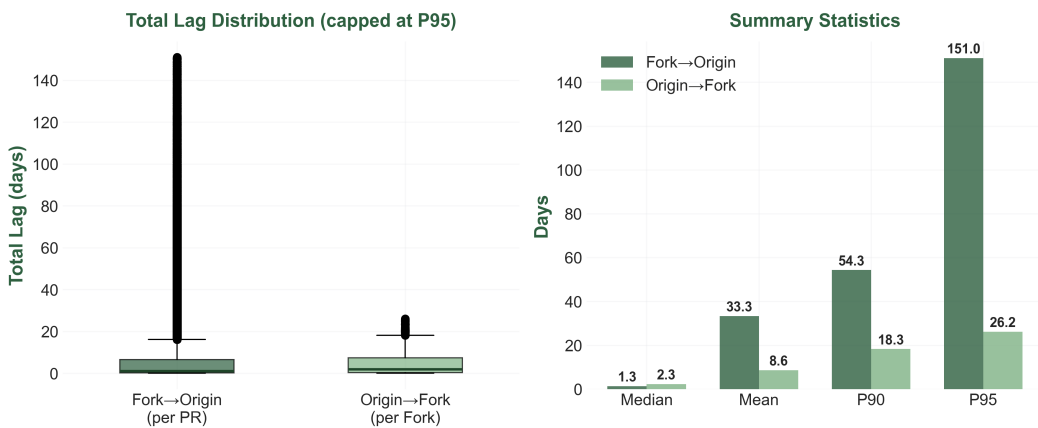


Fig. 6. Comparison of total synchronization lag by direction. Fork→Origin (developer batching + maintainer review per PR) vs. Origin→Fork (average pulling lag per fork).

lags exhibit pronounced long tails, with high upper quantiles and extreme maxima, consistent with occasional but substantial slowdowns that dominate average delay.

Activity does not prevent drift: Surprisingly, pulling lag is not confined to inactive forks. We observe a positive association between pull activity and pulling lag ($\rho = 0.508$, $p < 0.001$): forks with more pull operations tend to have higher average delays. Concretely, we identify 414 forks with more than 1,000 pull operations yet more than 10 days of average pulling lag, demonstrating that high synchronization activity does not guarantee low drift. This suggests that drift can persist even under frequent integration attempts, likely due to bursty update patterns, delayed adoption of specific upstream commits, or selective pulling strategies.

Directional contrast in total lag: Figure 6 compares direction-specific lag distributions. Although medians are comparable, fork→origin synchronization exhibits a substantially heavier tail than origin→fork pulling, revealing a *propagation asymmetry*: even when upstream code successfully merges, it may take weeks to propagate back through the fork ecosystem. This has direct security implications—patches merged into origins may remain unapplied in downstream forks, leaving the software supply chain exposed.

Commit-level lifecycle confirms fork pulling as the bottleneck: To validate these PR-level findings, we traced the end-to-end lifecycle of 33,767 individual commits from creation through upstream merge to adoption by the last synchronized fork. The median total lifecycle is 3.17 days

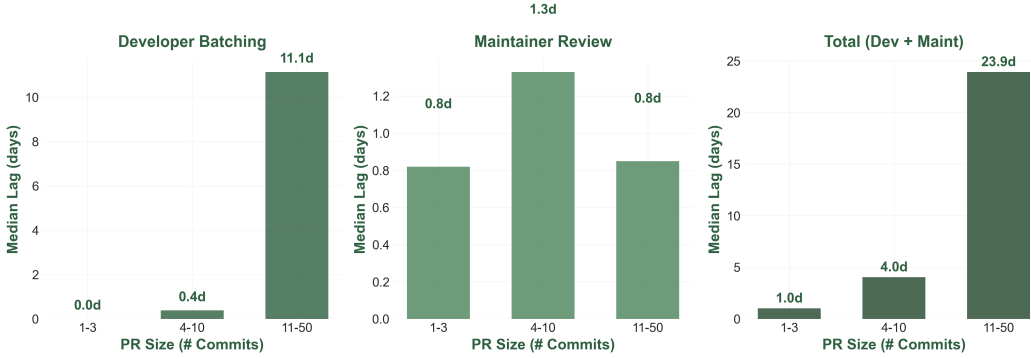


Fig. 7. Fork→Origin: Median lag by PR size. Developer batching lag increases substantially with PR size, while maintainer review lag remains relatively constant.

(90th percentile: 50.48 days). Critically, the Fork Pulling Phase accounts for **72.9%** of the entire life cycle, despite the Developer Batching and Maintainer Review Phases involving explicit human coordination. Moreover, 38.8% of commits experience pulling lag, nearly 3× higher than the 13.7% review lag, confirming that fork pulling is the most consistently delayed phase in the pipeline.

Finding 3: Synchronization delay is pervasive and structurally uneven. The Fork Pulling Phase accounts for 72.9% of end-to-end delay, with 38.8% of commits experiencing pulling lag—nearly 3× the 13.7% affected by maintainer review. Fork pulling is the dominant bottleneck, creating persistent vulnerability windows.

PR size drives batching lag: Figure 7 shows batching-lag prevalence rises from 4.3% for single-commit PRs to 57.2% for PRs with 11–30 commits (13× increase), while review latency shows no such correlation. This implies that large PRs act as a temporal filter—disproportionately delayed before submission, reducing the likelihood of timely upstream exposure. Because batching lag is predictable from PR size, tools can intervene early by recommending PR decomposition and prompting earlier submission of partial units.

4.4 RQ3: Pull Request Categorization

RQ1 and RQ2 established that synchronization may be either delayed or incomplete. However, these findings do not explain what kinds of changes successfully synchronize and why synchronization attempts fail. Understanding these patterns is essential for designing effective interventions. In this RQ, we therefore categorize the intent of merged PRs to identify which contribution types propagate successfully, and analyze rejected PRs to reveal the root causes of synchronization failure.

4.4.1 RQ3.1: Categorization of Intentions. To analyze cross-repository contributions from forks to origin repositories, we collected **158,482** closed and merged PRs submitted by forks to their corresponding origin repositories. To prioritize PRs that are more likely to reflect meaningful and sustained parallel development, we rank PRs in descending order based on the popularity of the contributing forks, measured by their star counts, which have been widely used as a proxy for project maturity and community interest. From this ranked list, we selected the top **30,000** PRs for in-depth analysis to match our analysis capacity. These PRs were further examined to infer their functional intent using a hybrid card sorting approach [17, 46] with the aid of an LLM and human consolidation to dynamically derive the categorizations.

We first derived an initial set of high-level intent categories from established taxonomies of software changes and pull-request studies, including bug fixing, feature addition, refactoring, documentation, and performance optimization according to related commit classification work [20, 23, 38]. These categories reflect commonly accepted dimensions of developer intent in collaborative software evolution and provide a stable conceptual starting point.

Next, we employed an LLM, DeepSeek-R3.2 [14], to support the systematic classification of merged PRs by intent, following recent work demonstrating the effectiveness of LLMs for software engineering tasks [24]. For each PR, the LLM was provided with multiple information sources, including the PR title, description, commit messages, and code diffs, and was prompted to assign the PR to the most appropriate intent category. In addition, the model was instructed to explicitly flag PRs whose changes could not be adequately captured by any existing category.

We constructed the intent taxonomy using a hybrid card-sorting process that combines both inductive and deductive reasoning. Specifically, we began with a small set of high-level, pre-defined categories derived from prior studies on code change classification and software maintenance activities. Using a randomly sampled subset of PRs, we then performed an open coding phase, in which the LLM proposed fine-grained intent labels based on observed change patterns without being restricted to the initial taxonomy. These emergent labels were subsequently reviewed and consolidated into candidate categories.

Lastly, we applied the evolving taxonomy to the full PR dataset. Whenever the LLM consistently identified PRs that did not fit well into existing categories, we revisited the taxonomy and refined it through category splitting or merging, guided by semantic coherence and empirical frequency. This iterative refinement continued until the taxonomy stabilized and no recurrent uncategorizable PRs were observed.

The resulting taxonomy has 37 categories and satisfies three design criteria: (i) categories are mutually exclusive, such that each PR is assigned to exactly one category; (ii) categories are collectively exhaustive with respect to the analyzed PRs; and (iii) category definitions are interpretable at the individual PR level. Each PR was ultimately assigned to the single category that best reflects its dominant intent. The full prompting protocol and taxonomy definitions are publicly available on our project website [13].

4.4.2 Categories of Intentions. Our analysis reveals the distribution of the most frequent PR categories. Bug fixes dominate fork-to-origin contributions, accounting for **36.42%** (10,927 PRs) of all analyzed PRs. This is followed by feature additions (**14.15%**), code refactoring (**8.63%**), documentation updates (**5.79%**), and performance optimizations (**5.27%**). Together, the top five categories constitute over **70%** of all PRs, indicating that the majority of fork contributions focus on maintenance and quality improvement rather than novel functionality.

Beyond these dominant categories, the remaining PRs form a long tail of diverse contribution types, including platform-specific modifications, dependency upgrades, configuration changes, and localization. Although each of these categories individually represents a small fraction of PRs, their collective presence highlights the heterogeneous nature of parallel development across forks.

For analytical clarity, we further grouped the 37 fine-grained categories into higher-level thematic clusters as in Figure 8. For clarity, the categories with fewer than 100 PRs were omitted. Bug and security-related fixes—including bug fixes, compatibility fixes, compiler fixes, and safety enhancements—constitute approximately **41.99%** of all PRs. Feature development accounts for **17.76%**, while code quality improvements (e.g., refactoring and optimization) represent **9.59%**. Documentation and testing-related changes comprise **8.88%**, and build or deployment-related PRs account for **8.50%**. Finally, customization-oriented changes, such as platform- or project-specific modifications, represent **7.42%** of PRs.

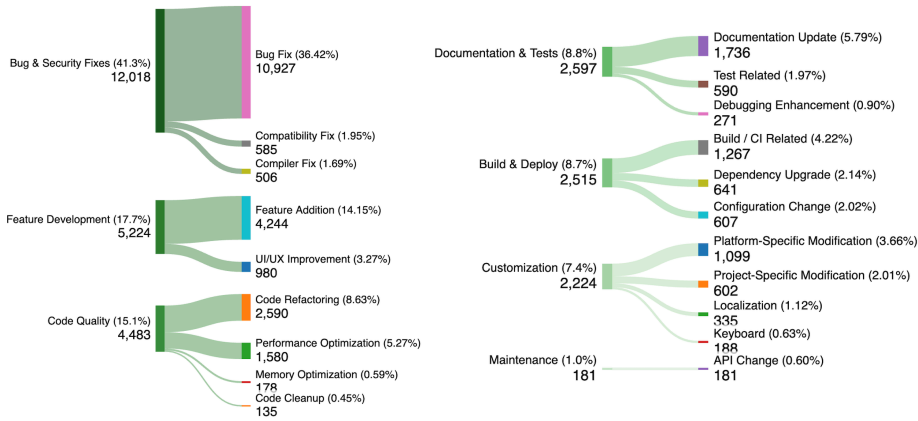


Fig. 8. Thematic Grouping of PR Intention Categories

Finding 4: Fork-to-origin PRs are primarily maintenance-driven: bug fixes alone account for 36% of contributions, and bug/security-related fixes collectively represent 42%. Over 70% of PRs focus on maintenance and quality improvement—including refactoring, optimization, compatibility, and documentation—rather than new features (17.8%). This concentration suggests that forks serve as a significant source of corrective and stabilizing contributions to upstream projects.

4.4.3 RQ3.2: What Gets Rejected and Why. RQ1 revealed a synchronization paradox: although approximately 90% of submitted PRs are merged, only a small fraction of fork commits ever appear in PRs. To explain what prevents synchronization when it is attempted, we analyze a sample of 5,000 rejected fork→origin PRs and examine the reasons they fail to synchronize. As discussed previously, we categorize rejection reasons using the same LLM-assisted hybrid card-sorting procedure. Unlike the intent analysis for merged PRs, this classification focuses on rejection rationales inferred from PR discussions and review comments, and allows multiple reasons to be assigned to a single PR.

Our analysis reveals the distribution of rejection categories. Contrary to common assumptions, rejection is *not* primarily driven by technical incorrectness. Only 17.3% involve any technical concerns. Instead, rejections concentrate in three dominant categories.

First, *superseded or duplicate* PRs account for 24.6% of all rejections. These PRs implement functionality that has already been merged through another contribution, indicating that multiple forks often pursue similar solutions independently. This pattern reflects the parallel development observed in RQ1: when synchronization is delayed or uncoordinated, redundant solutions emerge.

Second, *process issues* account for 24.1% of rejections. These PRs are typically technically plausible but violate contribution norms, such as submitting development branches instead of curated changes, including excessive or noisy commit histories, or lacking required formatting. Notably, these PRs are substantially larger than other rejected PRs in terms of the number of commits (mean 47.3 commits versus 11.4 for duplicates), indicating that process overhead is a significant barrier to synchronization rather than code quality.

Third, *maintainer policy decisions* explain 16.7% of rejections. These PRs are actively reviewed (98.8% have maintainer comments) but rejected due to misalignment with project direction, architectural vision, or roadmap priorities. Unlike process-related rejections, these decisions represent structural limits to synchronization that cannot be easily resolved through revision.

Together, these three categories account for **65.4%** of all rejected PRs. The remaining rejections are distributed across insufficient information (11.7%), code quality issues (10.5%), project fit mismatch (6.2%), and minor categories such as incorrect submission targets or communication failures. Rejection patterns are also highly concentrated: the top ten origin repositories account for **50.8%** of all rejected PRs, suggesting that synchronization barriers are strongly shaped by project-specific norms rather than universal rules.

Finding 5: PR rejection is rarely caused by technical incorrectness—only **17.3%** of rejected PRs involve any technical concerns. Instead, over **65%** of rejections stem from superseded contributions (24.6%), process violations (24.1%), or maintainer policy decisions (16.7%), indicating that synchronization failures are driven by coordination overhead and governance constraints rather than code quality.

4.5 RQ4: Synchronizable Commits Mining

RQ1–RQ3 show that synchronization is selective and often blocked by non-technical barriers. RQ4 shifts focus from PRs to fork-local commits, asking how much divergence could be avoided by identifying sync-ready commits that remain local. The platform’s design point is visibility: synchronization gaps are typically invisible to downstream users and maintainers by default, and surfacing them as optional recommendations allows informed decisions without imposing automated cross-repository pushes.

Concretely, we focus on fork-local unique commits that satisfy two principles mainly derived from previous conclusions. First, they are **sync-worthy**: the change is broadly beneficial to other repositories (e.g., bug fixes, compatibility improvements, performance optimizations) and is not merely fork-specific new functionality tied to a local codebase. Second, they are **sync-eligible**: the change is technically portable to the target repository (or peer forks) and does not violate repository-specific constraints such as project scope, contribution rules, or architectural invariants. These criteria motivate our staged syncability assessment pipeline, which operationalizes technical portability through conservative checks and usefulness verification.

Design goals: Our assessment targets technical mergeability rather than social acceptance. Accordingly, the pipeline is (i) conservative: false positives are minimized; (ii) stage-gated: later checks run only if earlier checks pass; (iii) decision-based: binary pass/fail rather than weighted scoring. To increase robustness against fork divergence, we compare candidates not only against the origin repository but also against other forks in the family, enabling multi-repository evidence for file availability and diff-context compatibility. A candidate is labeled Synchronizable if it passes the following stages:

Stage 0 Candidate extraction and prioritization. We extract all native commits with metadata and diffs, and identify the test files by keywords to treat them separately. The following pipeline prioritizes commits that include test files, as these tend to be better packaged and easier to validate.

Stage 1 Criteria of Sync-worthy.

- (1) **Large-scale refactoring filter** rejects commits that touch too many files (default threshold: ≥ 50 files). Based on the findings from RQ3, we first sort candidates by the number of code files modified (ascending) and apply this filter because large, cross-cutting refactorings are typically difficult to port across repositories: they require substantial review effort, are more likely to conflict with repository-specific evolution, and often demand non-trivial manual conflict resolution and re-validation.
- (2) **Dependency-only update filter** rejects commits whose modified files are exclusively dependency or third-party management (e.g., build manifests, lock files, and vendor/ or third_party/

directories). We apply this filter because dependency configurations are often *project- and environment-specific*: forks may pin different versions, enable different feature flags, select different build options, making such changes unlikely to be directly transferable across repositories.

- (3) **LLM-based usefulness screening**: given the commit message and a structured diff summary, an LLM predicts whether the change is broadly beneficial beyond the local fork. This screening is grounded in the empirical evidence from RQ3: the intent taxonomy of merged PRs identifies the change types that are generally valued beyond the current fork (e.g., bug fixes, compatibility fixes, refactoring/quality improvements, and performance optimizations). As long as the commit falls into the beneficial category, it passes the screening. Note that the usefulness of certain commits varies across repository context, and it is very hard for non-maintainers to accurately determine whether a commit is useful for a given repository without understanding the whole code context. Therefore, we do not take the screening of context-dependent usefulness into account.

Stage 2 Criteria of Sync-eligible. Stage 2 performs a conservative, stepwise assessment of whether a fork-local commit can be ported to a target repository (the origin and/or peer forks). We follow the taxonomy of patch-porting difficulty [53, 59]: Type-I (direct application, no adaptation needed), Type-II (patch-location changes due to file renaming or restructuring), Type-III (symbol resolution and dependency importing), and Type-IV (significant code transformations and module-level modifications). We conservatively operationalize sync-eligibility as covering only Type-I–II cases, and defer Type-III–IV (which requires substantial semantic adaptation and is empirically harder to automate) to future work.

- (1) **LLM-assisted Guideline compliance screening**: Before attempting technical porting, we use an LLM to assess whether the change is likely to violate the target repository’s contribution constraints (e.g., required formatting, test expectations, and scope rules) by jointly examining CONTRIBUTING.md/CI guidance when available, the commit message, and a structured diff summary. This step reduces unnecessary commits that might be predictably rejected or impose avoidable review burden.
- (2) **Patch-location & context feasibility (Type-I–II)**. We first verify that all modified file paths exist in the latest target repository snapshot, and then validate that each diff hunk’s pre-change context is present for Type-I–II porting. Specifically, the context of all hunks must match the target files under normalized namespace, approximating the minimal feasibility condition for Type-I–II porting: a valid landing location and a recognizable pre-change context in the target codebase. If file paths are missing or hunk contexts cannot be matched, applying the change would likely require more intrusive re-targeting, which is characteristic of Type-III–IV or higher-complexity porting and is conservatively excluded in our study.

4.5.1 Effectiveness of Locating Synchronizable Candidates. Applying the pipeline to 3,820 forks yielded 5.4 million native commits. Stage 0 prioritized commits by ranking them in ascending order of non-test file count, retaining the top 0.5 million commits that include test files. This threshold was chosen as a practical scope for a proof-of-concept evaluation; the pipeline itself is designed to scale to the full corpus in future work. Technical eligibility checks in Stages 1–2 further reduce candidates to 24,116 pairs, removing commits incompatible with target repositories. Domain filtering excludes 2,087 pairs touching only templates or tests and 5,320 pairs without source files, leaving 16,709 pairs. Finally, intent screening restricts to sync-worthy categories (Bug Fix, Security Fix, Code Quality), yielding the final **12,284** sync-ready pairs, resulting in 2.45% of the sampled 0.5 million.

Pull request submission to fork families. Our tool identified **12,284** sync-ready commit–repository pairs. To validate practical value, we have submitted **73** pull requests to target repositories (origins or peer forks), selecting candidates where the change is expected to be useful and acceptable. Scaling beyond this sample was infeasible: GitHub’s anti-abuse mechanisms flagged and restricted

our accounts after detecting automated cross-repository PR activity. As of this writing, 3 PRs have been merged while the rest remain under review or discussion. The monitoring platform is currently deployed and actively tracking synchronization opportunities across the collected fork families in our dataset, with new sync-ready candidates detected within hours of commit creation. To enable broader adoption, we plan to release the platform publicly, allowing developers to opt in and receive sync recommendations for their own fork families [13].

Supply-chain risk and maintainer control. Automatically pushing fork-derived changes (as bulk PRs or notifications) to maintainers or to peer forks would invite supply-chain risk and is the scenario that GitHub’s anti-abuse mechanisms guard against. The platform intentionally avoids this: candidate sync-worthy commits are surfaced as optional recommendations that maintainers see only when they choose to look. The platform thereby improves the exposure of syncable commits, including stale security fixes that failed to propagate upstream.

Operational cost. The platform’s operational overhead is modest. For the final security-commit pipeline, LLM API cost is \$0.01–0.04 per commit, and end-to-end processing time is 2.2–5.4 seconds per commit. The full one-time experiments, including preliminary testing, cost approximately \$350.

Security validation of syncable commits. To assess whether unsynced commits expose target repositories to concrete security risks, we manually validated 153 sync-ready security-related commit–repository pairs from our dataset, covering ecosystems such as Bitcoin, Ceph, OpenWrt/LEDE, ClickHouse, Dogecoin, and others. For each pair, we fetched the commit from GitHub, identified the security-relevant change, and then checked whether the target repository still contained the vulnerable or outdated code. Of the 153 pairs, 83 were manually reviewed and confirmed as potential 1-day vulnerabilities: the target repository had not applied the fix and remained exposed. These span diverse security categories including patching with known CVEs (e.g., Dropbear SSH, glibc, miniupnpc), build hardening gaps (missing compiler flags, unnecessary sample code), stale sanitizer suppressions masking real bugs, and insecure CI/Docker configurations. We have manually produced 35 proof-of-concept so far to demonstrate these vulnerabilities and filed issues to the affected repositories to draw maintainer attention. Continuous progress on these security findings is tracked on our monitoring platform [13].

Manual validation of sync-ready candidates. To verify that the LLM-based screening is reliable, two authors independently reviewed a random sample of 300 sync-ready candidates drawn from the full pipeline output, with a third resolving disagreements [52]. Of the 300 candidates, 258 (86%) were judged genuinely sync-worthy with 85% inter-rater agreement [10], confirming that the LLM screening aligns well with human judgment.

5 Related Work

5.1 Forking Behavior and Parallel Development

Prior fork studies establish forking motivations [12, 26], classification criteria that distinguish sustained hard forks from short-lived PR-only forks [65, 66], ecosystem mappings and tooling [11, 48], distributed-collaboration foundations within single projects [37], and narrow security-propagation effects [31]. Closest to our angle, Lin et al. study how Linux distributions coordinate bug fixes and security patches with their upstream projects [33, 34], but in a curated package-level setting (Debian, Fedora) with formal integrator roles. We instead study general GitHub fork families, including independently evolving and unmaintained forks, at the commit level, with bidirectional synchronization, lag decomposition, rejection-barrier analysis, and sync-ready missed-opportunity mining as the focus. None of these prior works analyze bidirectional commit synchronization or quantify technical lag across fork families.

5.2 Pull Request Studies

The pull-based development model is well-studied. Rigby et al. [49] characterize open-source peer review as early, frequent reviews of small, independent contributions; our size-effect on batching lag corroborates this in the fork→origin setting. Gousios et al. [20, 22] explore pull-based development and the integrator's perspective. Zhang et al. [62, 63] analyze merge-decision factors and PR latency, and Bacchelli and Bird [4] study modern code-review practice. Prior studies of patches, pull requests, and modern code review [16, 27, 29, 54, 58, 61, 62] characterize merge decisions, latency, rejection, abandonment, and review friction, with a mix of technical, process, social, and project-context factors. None of these address fork-family synchronization specifically.

5.3 Patch Porting and Commit Migration

Another line of work examines how patches and commits are reused, duplicated, or ported across related codebases. Li et al. [32] investigated patch porting practices in the Linux kernel ecosystem and showed that patches are frequently backported across branches with varying strategies and delays. Nguyen et al. [44] demonstrated that similar patches often appear in multiple locations due to cherry-picking [9] or manual migrations, and code clone detection techniques [50, 51] have been applied to identify such duplicated changes. Pan et al. [47] proposed PPatHF, an LLM-based approach for automating zero-shot patch porting across hard forks, demonstrating that 42.3% of patches from Vim could be automatically ported to Neovim, highlighting both the feasibility and the remaining difficulty of cross-fork patch migration. Merge conflicts [8, 18] further complicate synchronization as codebases diverge. Hu et al. [25] studied the patch propagation in the Golong ecosystem. However, prior work focuses on branches within a single project or on specific fork pairs; the broader fork ecosystem on GitHub, where commit synchronization spans over parallel development, remains far less understood.

6 Discussion

6.1 Threats to Validity

6.1.1 External Validity. Platform scope (GitHub only). Our study focuses on GitHub repositories to enable consistent cross-repository commit tracking and to leverage rich contribution metadata (e.g., PR discussions, merge outcomes, and review timelines) [5, 28]. This choice may bias findings toward GitHub-specific development practices. Nevertheless, GitHub hosts a diverse range of projects and provides PR-centric signals that are difficult to recover from broader archival sources (e.g., Software Heritage) that lack comparable contribution and review process data.

Fork-family construction. We identify actively maintained forks using observable signals such as stars and commit activity. These indicators may misclassify some forks (e.g., actively used but low-star forks, or periodically maintained forks with sparse commits). However, given the absence of reliable ground truth for "active maintenance" at scale, following the prior work, these signals provide a practical and reproducible approximation for deriving fork families.

Temporal bias. Our analysis is based on a snapshot of repository data collected at a fixed point. Synchronization activity that occurred after our data collection window, or commits that were synchronized shortly before our snapshot but not yet propagated to all forks, may be missed. This limitation is inherent to empirical studies of evolving repositories; we mitigate it by analyzing long observation windows and focusing on structural patterns rather than point-in-time counts.

Selection bias toward popular repositories. Our 2,000 star-ranked origins are concentrated in the popular end of the GitHub distribution (rationale in Section 3), so synchronization patterns measured here may not transfer to long-tail projects with weaker maintainer engagement and smaller fork ecosystems. Headline numbers (90% merge rate, 65% governance-dominated rejection

mix, 72.9% pulling-lag share) could shift for the unselected projects, and quantitative magnitudes such as CER values and lag durations should be read as estimates conditional on the popular projects. We mitigate this in two ways: we report medians and long-tail distributions rather than only means; we ground qualitative findings in mechanisms that do not depend on popularity.

6.1.2 Internal Validity. LLM-based categorization and taxonomy refinement. Our intent and rejection-reason analyses rely on LLM-assisted hybrid card sorting, which may introduce misclassification. We mitigate this by manually validating 300 random sync-ready commit–repository pairs (two independent reviewers, a third resolving disagreements), yielding **85%** inter-rater agreement and **86%** confirmed sync-worthy (Section 4.5.1), and by using a validator-role setup to reduce systematic LLM errors. Our policy-compatibility screening likewise relies on LLM interpretation of repository guidelines, which we partially validate against real PR submission behavior.

Subjectivity of manual sync-worthiness validation. The human review reflects author judgment rather than upstream-maintainer acceptance, and the sample ($\approx 2.4\%$ of 12,284 candidates) bounds the precision of the estimate. However, considering the difficulty in obtaining the ground truth from upstream maintainers, and manual efforts, we only sampled 300 candidates for validation, and release the annotated sample for independent audit.

Syncability pipeline assumptions and false negatives. Our syncability assessment is intentionally conservative and may exclude commits that are technically portable but require non-trivial adaptation, increasing false negatives. We prioritize high-confidence, sync-worthy candidates over noisy recommendations, using prioritization rather than hard thresholds and reserving hard assumptions for technical eligibility checks. Our current operationalization targets Type-I–II porting; more complex cases remain hard to automate robustly and are left for future work.

Limited PR submission validation. We validated the pipeline by submitting 73 PRs to target repositories. Although small relative to the 12,284 sync-ready pairs (0.6%), scaling up was infeasible: GitHub’s anti-abuse mechanisms flagged and restricted our accounts after detecting automated cross-repository PR activity, even with authenticated identities, reflecting a tension between large-scale synchronization research and anti-spam policies. To address this, we progressively release our monitoring platform publicly so developers can opt in and receive sync recommendations for their own fork families rather than unsolicited PRs.

6.2 Discussion: Lessons Learned

Lesson 1: Effective synchronization tooling must be governance- and value-aware, not merely technically feasible. Our rejection analysis (Finding 5, Section 4.4.3) shows that over 65% of rejected PRs are blocked by process violations, maintainer policy decisions, or redundancy rather than technical defects, while RQ4 (Section 4.5) reveals that many fork-local commits are mechanically portable yet offer limited value or conflict with target-repo constraints. Together, these findings imply that synchronization tools must go beyond patch applicability to jointly consider governance compatibility and semantic value. Our monitoring platform operationalizes this insight by filtering candidates through technical portability, usefulness grounded in empirically observed beneficial contribution types (Finding 4), and policy compatibility derived from repository guidelines. For practitioners, this reframes synchronization risk: investing solely in better patch application is insufficient if submissions systematically violate norms. For the research community, our evidence base makes implicit governance barriers explicit, motivating policy-awareness as a first-class design criterion for synchronization tools.

Lesson 2: Rebased or amended PR commits make SHA-only exposure metrics conservative. Our classification of 532,070 commits in 158,472 fork→origin PRs (RQ1.2, Section 4.2.2) shows that 32.73% do not match any fork SHA but have a diff at least 85% similar to a fork unique commit,

indicating a rewritten identity from rebase or amend. At the PR level, 33.72% of PRs consist entirely of derivative commits. These derivative commits are not incidental noise; they reflect normal contribution preparation, such as rebasing on the latest upstream state or amending commits before opening a PR. Treating them as unexposed would misclassify synchronized changes as missing simply because the commit identifier changed. We therefore report a history-rewriting-aware CER alongside the exact-SHA baseline, separating the conservative lower bound from the content-aware synchronization estimate.

Lesson 3: Synchronization delay is dominated by fork pulling, not by PR review. Our commit-level lifecycle trace of 33,767 commits (Finding 3, RQ2, Section 4.3) shows that the Fork Pulling Phase accounts for 72.9% of end-to-end commit lifecycle delay, whereas developer batching and maintainer review together account for the remaining 27.1%. The prevalence of pulling lag (38.8% of commits) is roughly 3× the 13.7% prevalence of maintainer review lag. The high activity does not protect against drift: among 414 forks with more than 1,000 pull operations, the average pulling lag still exceeds 10 days. Tooling should therefore prioritize accelerating post-merge propagation which is exactly what our tool aims to address.

Lesson 4: A high PR merge rate can mask systemic under-exposure of fork-local work, a synchronization paradox invisible to PR-only metrics. Although 90% of submitted fork→origin PRs are merged, the median fork exposes only 7.19% of its post-fork commits through PRs at all (Finding 2, RQ1.2, Section 4.1.2), and roughly 2.9 million fork-local commits across our dataset never appear in any PR to the origin: a project can simultaneously sustain an excellent merge rate and a deeply unsynchronized fork ecosystem. The actionable consequence is that PR acceptance is the wrong primary signal for ecosystem-wide synchronization health. Tools and metrics targeting fork-family synchronization need the fork-level visibility that quantifies the unexposed commits and their syncability, rather than relying on PR acceptance rates alone. This is precisely the design point our monitoring platform operationalizes.

Lesson 5: Fork→origin rejections are mainly governance-driven, not technical. Prior patch, PR, and code-review studies [16, 27, 29, 54, 58, 62] show that merge outcomes reflect technical, process, social, and project-context factors. In our fork→origin setting, technical incorrectness accounts for only 1% of rejections, while process violations, maintainer-policy decisions, and superseded contributions account for over 65% (Finding 5, RQ3, Section 4.3). These failures often occur when upstream has already addressed the issue, the fork diverges from upstream scope, or the submission violates project norms. Synchronization tools should therefore prioritize project-specific PR preparation and avoid resubmitting policy-blocked or superseded changes.

7 Conclusion

This paper presents the first large-scale empirical study of fork synchronization across 2,000 GitHub origins, 3,820 forks, and 2.9 million fork-local commits. We find that only 7.19% of fork commits appear in PRs despite a 90% merge rate, that fork pulling accounts for 72.9% of synchronization delay, and that over 65% of PR rejections are governance-driven rather than technical. Guided by these insights, our three-stage syncability pipeline surfaces 12,284 sync-ready pairs (86% confirmed sync-worthy). Future work includes extending to Type-III–IV patch porting and deploying the monitoring platform for community adoption.

Acknowledgments

This research is supported by the National Research Foundation Singapore, Prime Minister’s Office, Singapore, and the Cyber Security Agency under the National Cybersecurity R&D Programme (NCRP25-P04-TAICeN) and its Campus for Research Excellence and Technological Enterprise (CREATE) programme.

Data Availability

Our anonymized dataset, analysis scripts, platform source code, and documentation describing the platform-construction process are available at <https://github.com/zly123987/fork-study>.

References

- [1] Sébastien Andreina, Lorenzo Alluminio, Giorgia Azzurra Marson, and Ghassan Karamé. 2023. Short Paper: Estimating Patch Propagation Times Across Blockchain Forks. In *Financial Cryptography and Data Security (Lecture Notes in Computer Science)*. Springer, 276–287. doi:10.1007/978-3-031-47751-5_16
- [2] Astral. 2025. astral-tokio-tar: Maintained fork of tokio-tar. GitHub. <https://github.com/astral-sh/tokio-tar>.
- [3] async-tar contributors. 2025. async-tar: A tar archive reading/writing library for async Rust. GitHub. <https://github.com/dignifiedquire/async-tar>.
- [4] Alberto Bacchelli and Christian Bird. 2013. Expectations, Outcomes, and Challenges of Modern Code Review. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. IEEE, 712–721. doi:10.1109/ICSE.2013.6606617
- [5] Christian Bird, Peter C. Rigby, Earl T. Barr, David J. Hamilton, Daniel M. German, and Premkumar Devanbu. 2009. The Promises and Perils of Mining Git. In *Proceedings of the 6th Working Conference on Mining Software Repositories (MSR)*. IEEE, 1–10. doi:10.1109/MSR.2009.5069475
- [6] Hudson Borges, Andre Hora, and Marco Tulio Valente. 2016. Understanding the Factors That Impact the Popularity of GitHub Repositories. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 334–344. doi:10.1109/ICSME.2016.31
- [7] Hudson Borges and Marco Tulio Valente. 2018. What’s in a GitHub Star? Understanding Repository Starring Practices in a Social Coding Platform. *Journal of Systems and Software* 146 (2018), 112–129. doi:10.1016/j.jss.2018.09.016
- [8] Yuriy Brun, Reid Holmes, Michael D. Ernst, and David Notkin. 2011. Proactive Detection of Collaboration Conflicts. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. ACM, 168–178. doi:10.1145/2025139.2025139
- [9] Panuchart Bunyakiati and Chadarat Phipathananunth. 2017. Cherry-Picking of Code Commits in Long-Running, Multi-release Software. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 896–900. doi:10.1145/3106237.3122818
- [10] Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46. doi:10.1177/001316446002000104
- [11] Valerio Cosentino, Javier Luis Cánovas Izquierdo, and Jordi Cabot. 2017. A Systematic Mapping Study of Software Development With GitHub. *IEEE Access* 5 (2017), 7173–7192. doi:10.1109/ACCESS.2017.2682323
- [12] Laura Dabbish, Colleen Stuart, Jason Tsay, and Jim Herbsleb. 2012. Social Coding in GitHub: Transparency and Collaboration in an Open Software Repository. In *Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW)*. ACM, 1277–1286. doi:10.1145/2145204.2145396
- [13] Dataset 2025. Dataset Website. <https://anonymous.4open.science/r/remediation-4E18/>.
- [14] DeepSeek 2026. DeepSeek. <https://www.deepseek.com/>.
- [15] edera-dev. 2025. CVE-2025-62518: TARmageddon (patches and reproducer repository). <https://github.com/edera-dev/cve-tarmageddon>. Accessed 2026-01-15.
- [16] Carolyn D. Egelman, Emerson Murphy-Hill, Elizabeth Kammer, Margaret Morrow Hodges, Collin Green, Ciera Jaspán, and James Lin. 2020. Predicting developers’ negative feelings about code review. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*. ACM, 174–185. doi:10.1145/3377811.3380414
- [17] Sally Fincher and Josh Tenenbergh. 2005. Making Sense of Card Sorting Data. *Expert Systems* 22, 3 (2005), 89–93. doi:10.1111/j.1468-0394.2005.00299.x
- [18] Gleiph Ghitto, Leonardo Murta, Marcio Barros, and Andre van der Hoek. 2018. On the Nature of Merge Conflicts: A Study of 2,731 Open Source Java Projects Hosted by GitHub. *IEEE Transactions on Software Engineering* 46, 10 (2018), 1155–1172. doi:10.1109/TSE.2018.2871083
- [19] GitHub. 2025. Syncing a fork. GitHub Docs. <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/syncing-a-fork>.
- [20] Georgios Gousios, Martin Pinzger, and Arie van Deursen. 2014. An exploratory study of the pull-based software development model. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. ACM, 345–355. doi:10.1145/2568225.2568260
- [21] Georgios Gousios and Diomidis Spinellis. 2012. GHTorrent: GitHub’s Data from a Firehose. In *Proceedings of the 9th Working Conference on Mining Software Repositories (MSR)*. IEEE, 12–21. doi:10.1109/MSR.2012.6224294
- [22] Georgios Gousios, Andy Zaidman, Margaret-Anne Storey, and Arie van Deursen. 2015. Work Practices and Challenges in Pull-Based Development: The Integrator’s Perspective. In *Proceedings of the 37th International Conference on Software*

- Engineering (ICSE)*. IEEE, 826–836. doi:10.1109/ICSE.2015.55
- [23] Abram Hindle, Daniel M. German, and Ric Holt. 2008. What Do Large Commits Tell Us? A Taxonomical Study of Large Commits. In *Proceedings of the 5th Working Conference on Mining Software Repositories (MSR)*. ACM, 99–108. doi:10.1145/1370750.1370773
 - [24] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* (2024). doi:10.1145/3695988
 - [25] Jinchang Hu, Lyuye Zhang, Chengwei Liu, Sen Yang, Song Huang, and Yang Liu. 2024. Empirical Analysis of Vulnerabilities Life Cycle in Golang Ecosystem. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3639230
 - [26] Jing Jiang, David Lo, Jiahuan He, Xin Xia, Pavneet Singh Kochhar, and Li Zhang. 2017. Why and how developers fork what from whom in GitHub. *Empirical Software Engineering* 22, 1 (2017), 547–578. doi:10.1007/s10664-016-9436-6
 - [27] Yujian Jiang, Bram Adams, and Daniel M. German. 2013. Will my patch make it? And how fast? Case study on the Linux kernel. In *Proceedings of the 10th IEEE Working Conference on Mining Software Repositories (MSR)*. IEEE. doi:10.1109/MSR.2013.6624016
 - [28] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. 2014. The Promises and Perils of Mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*. ACM, 92–101. doi:10.1145/2597073.2597074
 - [29] Oleksii Kononenko, Tresa Rose, Olga Baysal, Michael Godfrey, Dennis Theisen, and Bart de Water. 2018. Studying pull request merges: a case study of Shopify’s active merchant. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. ACM, 124–133. doi:10.1145/3183519.3183542
 - [30] Ravie Lakshmanan. 2025. TARMageddon Flaw in Async-Tar Rust Library Could Enable Remote Code Execution. <https://thehackernews.com/2025/10/tarmageddon-flaw-in-async-tar-rust.html>. Accessed 2026-01-15.
 - [31] Romain Lefeuvre, Charly Reux, Stefano Zacchiroli, Olivier Barais, and Benoit Combemale. 2025. Did You Forkget It? Detecting One-Day Vulnerabilities in Open-source Forks With Global History Analysis. (2025). arXiv:2511.05097 [cs.CR] doi:10.48550/ARXIV.2511.05097
 - [32] Xingyu Li, Zheng Zhang, Zhiyun Qian, Trent Jaeger, and Chengyu Song. 2024. An Investigation of Patch Porting Practices of the Linux Kernel Ecosystem. In *Proceedings of the 21st International Conference on Mining Software Repositories (MSR)*. 63–74. doi:10.1145/3643991.3644902
 - [33] Jiahuei Lin, Haoxiang Zhang, Bram Adams, and Ahmed E. Hassan. 2022. Upstream bug management in Linux distributions: An empirical study of Debian and Fedora practices. *Empirical Software Engineering* 27, 6 (2022), 134. doi:10.1007/s10664-022-10173-y
 - [34] Jiahuei Lin, Haoxiang Zhang, Bram Adams, and Ahmed E. Hassan. 2023. Vulnerability management in Linux distributions: An empirical study on Debian and Fedora. *Empirical Software Engineering* 28, 2 (2023), 47. doi:10.1007/s10664-022-10267-7
 - [35] Aravind Machiry, Nilo Redini, Eric Camellini, Christopher Kruegel, and Giovanni Vigna. 2020. SPIDER: Enabling Fast Patch Propagation in Related Software Repositories. In *Proceedings of the IEEE Symposium on Security & Privacy (S&P)*. 512–529. doi:10.1109/SP40000.2020.00038
 - [36] Addi Malviya-Thakur and Audris Mockus. 2024. The Role of Data Filtering in Open Source Software Ranking and Selection. In *Proceedings of the 1st IEEE/ACM International Workshop on Methodological Issues with Empirical Studies in Software Engineering*. 7–12. doi:10.1145/3643664.3648210
 - [37] Audris Mockus, Roy T. Fielding, and James D. Herbsleb. 2002. Two Case Studies of Open Source Software Development: Apache and Mozilla. *ACM Transactions on Software Engineering and Methodology* 11, 3 (2002), 309–346. doi:10.1145/567793.567795
 - [38] Audris Mockus and Lawrence G. Votta. 2000. Identifying Reasons for Software Changes Using Historic Databases. In *Proceedings of the International Conference on Software Maintenance (ICSM)*. IEEE, 120–130. doi:10.1109/ICSM.2000.883028
 - [39] Nuthan Munaiah, Steven Kroh, Craig Cabrey, and Meiyappan Nagappan. 2017. Curating GitHub for Engineered Software Projects. *Empirical Software Engineering* 22, 6 (2017), 3219–3253. doi:10.1007/s10664-017-9512-6
 - [40] National Vulnerability Database (NVD). 2019. CVE-2019-12735 Detail. <https://nvd.nist.gov/vuln/detail/CVE-2019-12735>. Accessed 2026-01-12.
 - [41] National Vulnerability Database (NVD). 2025. CVE-2025-62518 Detail. <https://nvd.nist.gov/vuln/detail/CVE-2025-62518>. Accessed 2026-01-15.
 - [42] Neovim Project. 2026. Neovim. <https://neovim.io/>
 - [43] Neovim Project. 2026. neovim/neovim (GitHub repository). <https://github.com/neovim/neovim>
 - [44] Hoan Anh Nguyen, Anh Tuan Nguyen, Tung Thanh Nguyen, Tien N. Nguyen, and Hridayesh Rajan. 2013. A Study of Repetitiveness of Code Changes in Software Evolution. In *Proceedings of the 28th IEEE/ACM International Conference*

- on *Automated Software Engineering (ASE)*. IEEE, 180–190. doi:10.1109/ASE.2013.6693078
- [45] Heise Online. 2025. Vulnerability discovered in Rust library for tar archives. <https://www.heise.de/en/news/Vulnerability-discovered-in-Rust-library-for-tar-archives-10794178.html>. Accessed 2026-01-15.
- [46] Optimal Workshop 2021. Hybrid Card Sorting. <https://support.optimalworkshop.com/en/articles/2626850-choose-between-an-open-closed-or-hybrid-card-sort>.
- [47] Shengyi Pan, You Wang, Zhongxin Liu, Xing Hu, Xin Xia, and Shanping Li. 2024. Automating Zero-Shot Patch Porting for Hard Forks. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 363–375. doi:10.1145/3650212.3652134
- [48] Luyao Ren, Shurui Zhou, and Christian Kästner. 2018. Forks Insight: Providing an Overview of GitHub Forks. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. ACM, 179–180. doi:10.1145/3183440.3195085
- [49] Peter C. Rigby, Daniel M. German, and Margaret-Anne Storey. 2008. Open source software peer review practices: a case study of the Apache server. In *Proceedings of the 30th International Conference on Software Engineering (ICSE)*. ACM, 541–550. doi:10.1145/1368088.1368162
- [50] Chanchal K. Roy and James R. Cordy. 2007. *A Survey on Software Clone Detection Research*. Technical Report 2007-541. Queen’s University.
- [51] Hitesh Sajjani, Vaibhav Saini, Jeffrey Svajlenko, Chanchal K. Roy, and Cristina V. Lopes. 2016. SourcererCC: Scaling Code Clone Detection to Big-Code. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. ACM, 1157–1168. doi:10.1145/2884781.2884877
- [52] Carolyn B. Seaman. 1999. Qualitative Methods in Empirical Studies of Software Engineering. *IEEE Transactions on Software Engineering* 25, 4 (1999), 557–572. doi:10.1109/32.799955
- [53] Ridwan Shariffdeen, Xiang Gao, Gregory J. Duck, Shin Hwei Tan, Julia Lawall, and Abhik Roychoudhury. 2021. Automated Patch Backporting in Linux (Experience Paper). In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 633–645. doi:10.1145/3460319.3464821
- [54] Yida Tao, Donggyun Han, and Sunghun Kim. 2014. Writing Acceptable Patches: An Empirical Study of Open Source Project Patches. In *Proceedings of the 30th IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 271–280. doi:10.1109/ICSME.2014.49
- [55] tokio-tar contributors. 2025. tokio-tar: A tar archive reading/writing library for async Rust on top of the Tokio runtime. GitHub. <https://github.com/vorot93/tokio-tar>.
- [56] Vim Project. 2026. Vim: The ubiquitous text editor. <https://www.vim.org/>
- [57] Vim Project. 2026. vim/vim (GitHub repository). <https://github.com/vim/vim>
- [58] Qingye Wang, Xin Xia, David Lo, and Shanping Li. 2019. Why is my code change abandoned? *Information and Software Technology* 110 (2019), 108–120. doi:10.1016/j.infsof.2019.02.007
- [59] Su Yang, Yang Xiao, Zhengzi Xu, Chengyi Sun, Chen Ji, and Yuqing Zhang. 2023. Enhancing oss patch backporting with semantics. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2366–2380. doi:10.1145/3576915.3623188
- [60] Xiao Yi, Yuzhou Fang, Daoyuan Wu, and Lingxiao Jiang. 2023. BlockScope: Detecting and Investigating Propagated Vulnerabilities in Forked Blockchain Projects. In *30th Annual Network and Distributed System Security Symposium, NDSS 2023*. The Internet Society. doi:10.14722/ndss.2023.24222
- [61] Lyuye Zhang, Chengwei Liu, Zhengzi Xu, Sen Chen, Lingling Fan, Lida Zhao, Jiahui Wu, and Yang Liu. 2023. Compatible Remediation on Vulnerabilities from Third-Party Libraries for Java Projects. In *Proceedings of the 45th International Conference on Software Engineering (Melbourne, Victoria, Australia)*. arXiv preprint arXiv:2301.08434, 2540–2552. doi:10.1109/ICSE48619.2023.00212
- [62] Xunhui Zhang, Yue Yu, Georgios Gousios, and Ayushi Rastogi. 2023. Pull Request Decisions Explained: An Empirical Overview. *IEEE Transactions on Software Engineering* 49, 2 (2023), 849–871. doi:10.1109/TSE.2022.3165056
- [63] Xunhui Zhang, Yue Yu, Tao Wang, Ayushi Rastogi, and Huaimin Wang. 2022. Pull request latency explained: an empirical overview. *Empirical Software Engineering* 27, 6 (2022), 126. doi:10.1007/s10664-022-10143-4
- [64] Zheng Zhang, Hang Zhang, Zhiyun Qian, and Billy Lau. 2021. An Investigation of the Android Kernel Patch Ecosystem. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 3649–3666. <https://www.usenix.org/conference/usenixsecurity21/presentation/zhang-zheng>
- [65] Shurui Zhou, Bogdan Vasilescu, and Christian Kästner. 2019. What the Fork: A Study of Inefficient and Efficient Forking Practices in Social Coding. In *Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE ’19)*. ACM, 350–361. doi:10.1145/3338906.3338918
- [66] Shurui Zhou, Bogdan Vasilescu, and Christian Kästner. 2020. How Has Forking Changed in the Last 20 Years? A Study of Hard Forks on GitHub. In *Proceedings of the 42nd International Conference on Software Engineering (ICSE ’20)*. ACM, 445–456. doi:10.1145/3377811.3380412

Received 2026-01-30; accepted 2026-06-25